Rideboard.ai Concept

07/12/2026 22:11:22

A Python backend makes sense for Atlas, especially because the product will eventually...

A Python backend makes sense for Atlas, especially because the product will eventually need AI orchestration, transportation-provider integrations, matching, pricing logic, and marketplace workflows.

I would use:

- **Next.js** for the web frontend
- **FastAPI** for the Python backend
- **PostgreSQL** later for persistent data
- **Redis/Celery or a queue** only when background jobs are actually needed

For the first prototype, you can still keep the UI mocked, but I would structure the repository so the Python backend is part of the plan from the beginning.

Recommended structure

```
atlas/
  apps/
    web/
      src/
      package.json

    api/
      app/
        main.py
        api/
          routes/
            health.py
            trips.py
            ride_options.py
        core/
          config.py
        models/
          trip.py
          ride_option.py
          community_driver.py
        services/
          trip_parser.py
          trip_planner.py
          recommendation_service.py
          providers/
            base.py
            mock_rideshare.py
```

```
            mock_transit.py
            mock_autonomous.py
            mock_community.py
        tests/
    pyproject.toml

docs/
README.md
```

Why FastAPI fits Atlas

FastAPI is a good fit because it gives you:

- Typed request and response models
- Automatic OpenAPI documentation
- Async support for calling provider APIs
- Clean separation from the frontend
- A strong foundation for future AI and marketplace services
- Python access to geospatial, ML, ranking, and optimization libraries

The backend should own transportation logic. The frontend should mainly collect input and present results.

Frontend responsibility

The Next.js app should handle:

- Trip conversation UI
- Form and message input
- Ride Board display
- Loading and error states
- Responsive design
- Client-side interaction state

Python backend responsibility

The FastAPI service should handle:

- Parsing a trip request
- Determining missing information
- Normalizing locations and times

- Querying transportation providers
- Combining provider results
- Ranking and recommending options
- Returning a consistent Ride Board response
- Eventually matching riders with community drivers

Initial API design

Parse a request

```
POST /api/v1/trips/parse
```

Request:

```
{
  "message": "I need to get from Potrero Hill to SFO tomorrow at 8 AM"
}
```

Response:

```
{
  "trip_request": {
    "origin": "Potrero Hill",
    "destination": "SFO",
    "departure_time": "2026-07-13T08:00:00-07:00",
    "passenger_count": null,
    "priority": null
  },
  "missing_fields": [],
  "next_question": null,
  "is_complete": true
}
```

Generate a Ride Board

```
POST /api/v1/ride-board
```

Request:

```
{
  "origin": "Potrero Hill",
  "destination": "SFO",
  "departure_time": "2026-07-13T08:00:00-07:00",
```

```
  "passenger_count": 1
}
```

Response:

```
{
  "recommended_option_id": "community-1",
  "options": [
    {
      "id": "rideshare-1",
      "type": "rideshare",
      "provider_name": "Uber / Lyft",
      "estimated_duration_minutes": 24,
      "estimated_cost_min": 34,
      "estimated_cost_max": 49,
      "recommendation_label": "Fastest"
    }
  ]
}
```

Pydantic models

```
from datetime import datetime
from enum import StrEnum

from pydantic import BaseModel, Field


class TripPriority(StrEnum):
    FASTEST = "fastest"
    CHEAPEST = "cheapest"
    BEST_VALUE = "best-value"
    FEWEST_TRANSFERS = "fewest-transfers"
    MOST_COMFORTABLE = "most-comfortable"


class TripRequest(BaseModel):
    origin: str | None = None
    destination: str | None = None
    departure_time: datetime | None = None
    arrival_time: datetime | None = None
    passenger_count: int | None = Field(default=None, ge=1, le=8)
    luggage_count: int | None = Field(default=None, ge=0)
    priority: TripPriority | None = None
    accessibility_needs: list[str] = Field(default_factory=list)
```

Provider abstraction

The backend should not know the details of Uber, transit, Waymo, or community rides directly. Give each provider the same interface.

```
from abc import ABC, abstractmethod

from app.models.ride_option import RideOption
from app.models.trip import TripRequest


class TransportationProvider(ABC):
    @abstractmethod
    async def get_options(
        self,
        request: TripRequest,
    ) -> list[RideOption]:
        raise NotImplementedError
```

Then create:

```
MockRideshareProvider
MockTransitProvider
MockAutonomousProvider
MockCommunityProvider
```

Later, those can be replaced with real integrations without rewriting the UI or planner.

Important architectural rule

Keep AI parsing separate from trip-planning logic.

For example:

```
User message
   ↓
Trip parser
   ↓
Structured TripRequest
   ↓
Trip planner
   ↓
Provider results
   ↓
Ranking service
```

```
    ↓
Ride Board response
```

That matters because an LLM should not be responsible for pricing, safety rules, provider eligibility, or core business logic. It can interpret language, but deterministic code should make product decisions wherever possible.

Revised MVP sequence

1. Build the static Ride Board in Next.js.
2. Create a small FastAPI service with `/health`.
3. Add mocked `/ride-board`.
4. Connect the frontend to the Python API.
5. Add deterministic trip parsing.
6. Add conversational follow-up logic.
7. Add tests around ranking and missing-field detection.
8. Add real integrations only after usability testing.

For your `technical-implementation.md`, I would revise the stack section to explicitly state that Atlas is a **Next.js frontend with a FastAPI backend**, even though the first backend responses will still use mocked provider data.

7/12/2026, 7:57:14 PM